```
Output:
I am a Car
I am also a Vehicle
```

If you remember when we looked at class context (static) methods, we used the keyword `self` in conjunction with the scope resolution operator `::` to access other static members. `parent` looks similar, doesn't it? There is an important difference; however, you might have already spotted it: `parent::__construct()` is not a static method.

A child calling or referencing a parent member will always use the scope resolution operator, so it resembles a static call even when accessing non-static members. The same restrictions apply as they do for `self`: you cannot access object context code from class context. While we're on the subject of accessing parent members and static context, there's something to be aware of…

Late Static Binding

Take a look at the following code; we've redesigned our `Vehicle`/`Car` class hierarchy a little. We'd like to use the DRY principle and not declare `whatAmI()` in every child class:

```
class Vehicle
{
    protected static string $iAm = 'Vehicle';

    public static function whatAmI(): void
    {
        print "I am a " . self::$iAm . PHP_EOL;
    }
}

class Car extends Vehicle
{
    protected static string $iAm = 'Car';
}
```

This code might look OK on first glance: we overload `$iAm` in the child class as before, but now we've moved the `whatAmI()` method into the parent `Vehicle` class. If we invoke `Car::whatAmI()`, it should use `Car::$iAm`, right?